MicroGoals: Documentation

SecureGoal Management Backend API

TEAM PROJECT ID: SWTID-2026-8363

TEAM SIZE: 4

TEAM LEADER:

- **NAME: ANU MITHRA.M**

Email: m.anumithra007@gmail.com

TEAM MEMBERS:

- **NAME: HARINI.K**

  Email: harinik1014@gmail.com

- **NAME: AMALA CHRISTY.S**

  Email: christyamala82@gmail.com

- **NAME: LAVANYA.R**

  Email: ramanvijayalakshmi65@gmail.com

Introduction

In the modern digital landscape, the ability to manage personal objectives efficiently and securely is paramount. **MicroGoals** is a robust, RESTful backend solution specifically engineered to meet these needs. By leveraging a modern technology stack, this

project provides a reliable infrastructure for users to organize their goals while maintaining the highest standards of data privacy and system integrity. The following documentation outlines the architectural decisions, security protocols, and core functionalities that make MicroGoals a scalable and secure environment for digital goal management.

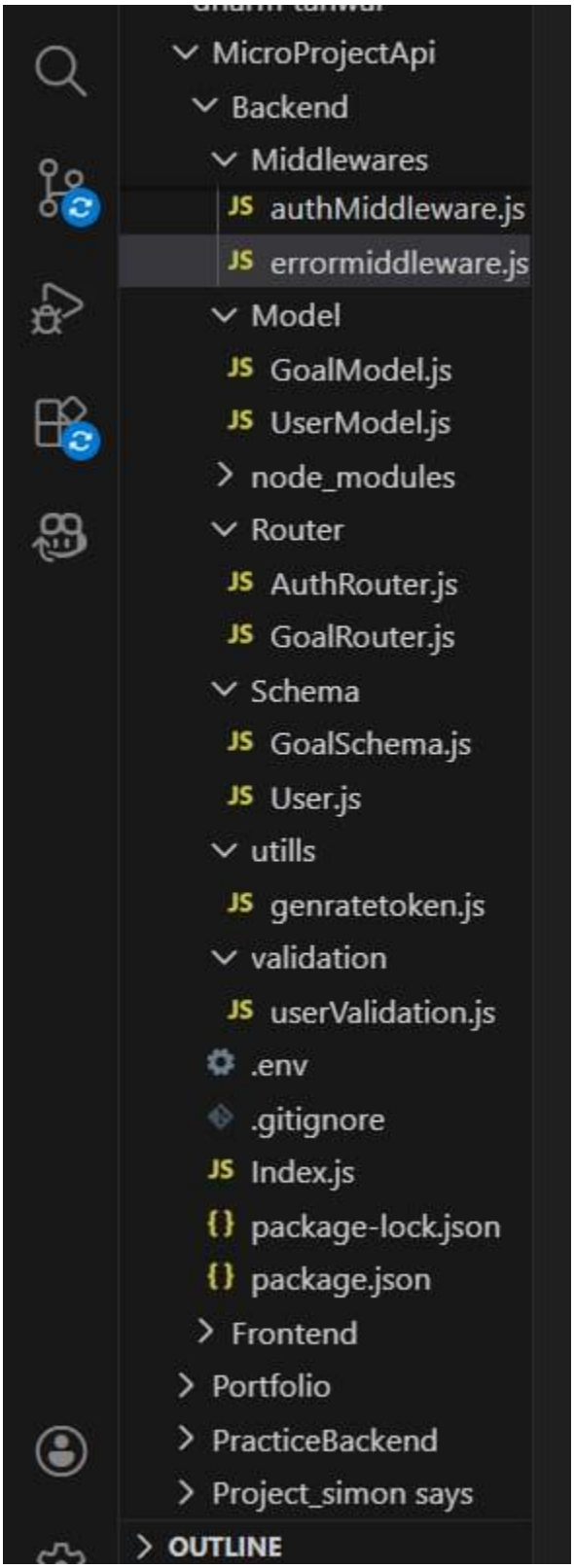
1. Project Overview

- MicroGoals is a secure, RESTful backend system designed for digital goal management. Built using the MVC (Model-View-Controller) architecture, it provides a scalable environment where users can securely register, log in, and manage their personal objectives. The system emphasizes data isolation, ensuring that users can only interact with the data they own.

2. System Architecture

- The project is built on a modular architecture to ensure maintainability and security:
- Runtime Environment: Node.js
- Web Framework: Express.js
- Database: MongoDB (NoSQL)
- Authentication: JSON Web Tokens (JWT)
- Data Modeling: Object Document Mapping (ODM) via Mongoose
- 📂 Logical Folder Structure

- Middleware: Houses the security logic (Authentication) and the global error-handling engine.
- Models/Schemas: Defines the data structure for Users and Goals.
- Routes: Manages the API endpoints and connects them to the logic.
- Configuration: Handles environment variables and database connectivity.


MicroProjectApi
Backend
Middlewares
authMiddleware.js
errormiddleware.js
Model
GoalModel.js
UserModel.js
node_modules
Router
AuthRouter.js
GoalRouter.js
Schema
GoalSchema.js
User.js
utills
genratetoken.js
validation
userValidation.js
.env
.gitignore
Index.js
package-lock.json
package.json
Frontend
Portfolio
PracticeBackend
Project_simon says
OUTLINE

Folder Structure

MicroGoals/

├── middleware/

│ ├── authMiddleware.js

│ └── errorMiddleware.js

├──Schema/

├──User.js

├──GoalSchema.js

├── models/

│ ├── userModel.js

│ └── goalModel.js

│

├── routes/

```
│   ├── authRoutes.js
│   └── goalRoutes.js
│
├── .env
├── Index.js
└── package.json
```

3. Database Design (ER Diagram Summary)

- The system uses a relational approach within a NoSQL database:
- User Entity: Stores identity details including Name, Email (Unique), and Hashed Passwords.
- Goal Entity: Stores the goal Title, Completion Status, and a Reference ID to the User.
- Relationship: One-to-Many. One user can create and manage multiple goals, but each goal belongs strictly to one user.

4. Key Features

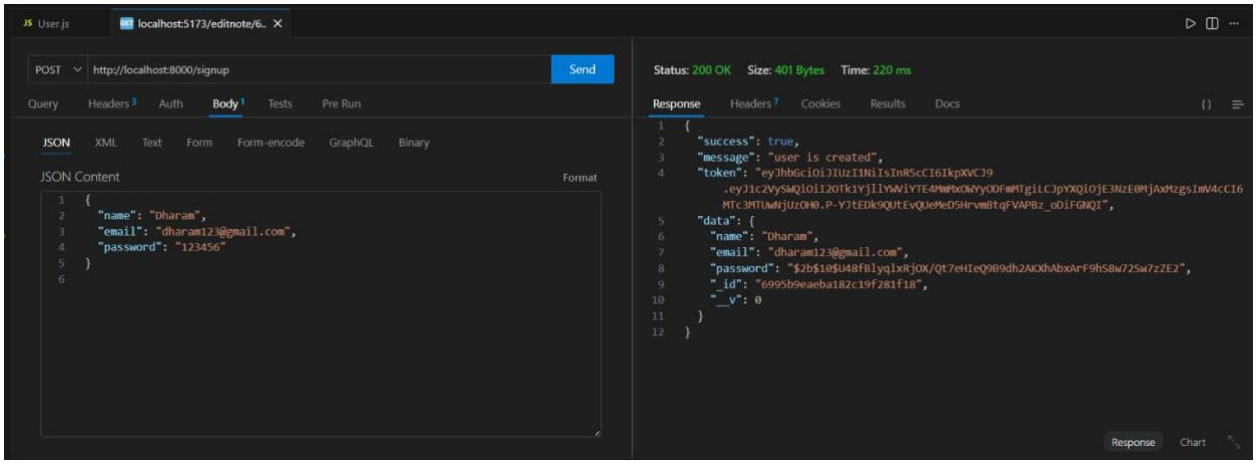
- 🔐 Secure Authentication
- User Onboarding: Registration and Login functionality.

- Password Security: Industry-standard hashing to ensure passwords are never stored in plain text.
- Token-Based Access: Upon login, the system generates a unique JWT. This token acts as a digital key for all subsequent requests.
- 📝 Goal Management (CRUD)
- Create: Users can add new goals to their personal dashboard.
- Read: Secure retrieval of goals, filtered specifically by the logged-in user's ID.
- Update: Ability to toggle the completion status of a specific goal.
- Delete: Permanent removal of goals from the database.
- 🛡️ Security & Validation
- Protected Routes: Only authenticated users with a valid token can access goal-related features.
- Data Ownership Validation: The system verifies that a user is the owner of a goal before allowing any updates or deletions.
- Centralized Error Handling: A unified system to catch and report errors, ensuring the API always returns a consistent and helpful response.

5. <u>User Workflow</u>

- Registration: User creates an account.
- Authentication: User logs in and receives a JWT.

- Authorization: The user includes the JWT in the header of their API requests.
- Interaction: The Middleware validates the token.
- Execution: The user performs CRUD operations on their personal goals.



6. Project Deliverables

- Functional API: A complete set of endpoints for user and goal management.
- Tested Endpoints: All routes verified via Thunder Client for reliability.
- Database Integration: Fully connected MongoDB Atlas/Local instance.
- Demonstration Video: A visual walkthrough of the registration, login, and goal management process

JS Index.js

```
require("dotenv").config();
const express = require("express");
const app = express();
const mongoose = require("mongoose");
const cors = require("cors");
app.use(cors());
const errormiddleware = require("./Middlewares/errormiddleware");
app.use(express.urlencoded({ extended: true }));
const bodyParser = require("body-parser");
app.use(express.json());
const PORT = process.env.PORT;
const Mongo_Url = process.env.MONGO_URL;
const authRoute = require("./Router/AuthRouter");
const GoalRouter = require("./Router/GoalRouter");

//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));

//route
app.use("/", authRoute);
app.use("/", GoalRouter);

app.use(errormiddleware);
app.listen(PORT, () => {
  console.log("Server running on port 8000");
});
```

7. Technology Stack Deep Dive

The system utilizes a modern “MERN-minus-React” stack to prioritize performance and security:

Node.js: Serves as the cross-platform JavaScript runtime environment for executing server-side code.

Express.js: Utilized as the web framework to design the RESTful routing and middleware integration.

MongoDB Atlas: A cloud-based NoSQL database used for flexible data modeling and high availability.

Mongoose (ODM): Provides a schema-based solution to model application data and manage relationships between Users and Goals.

8. API Endpoints Specification

The project exposes a series of RESTful endpoints to manage the application lifecycle:

Authentication Routes: Includes /api/users/register for account creation and /api/users/login for credential verification.

Goal Management Routes: Includes POST /api/goals for creation, GET /api/goals for retrieval, and DELETE /api/goals/:id for removal.

Status Updates: A specific PUT or PATCH route for toggling the completion status of a goal.

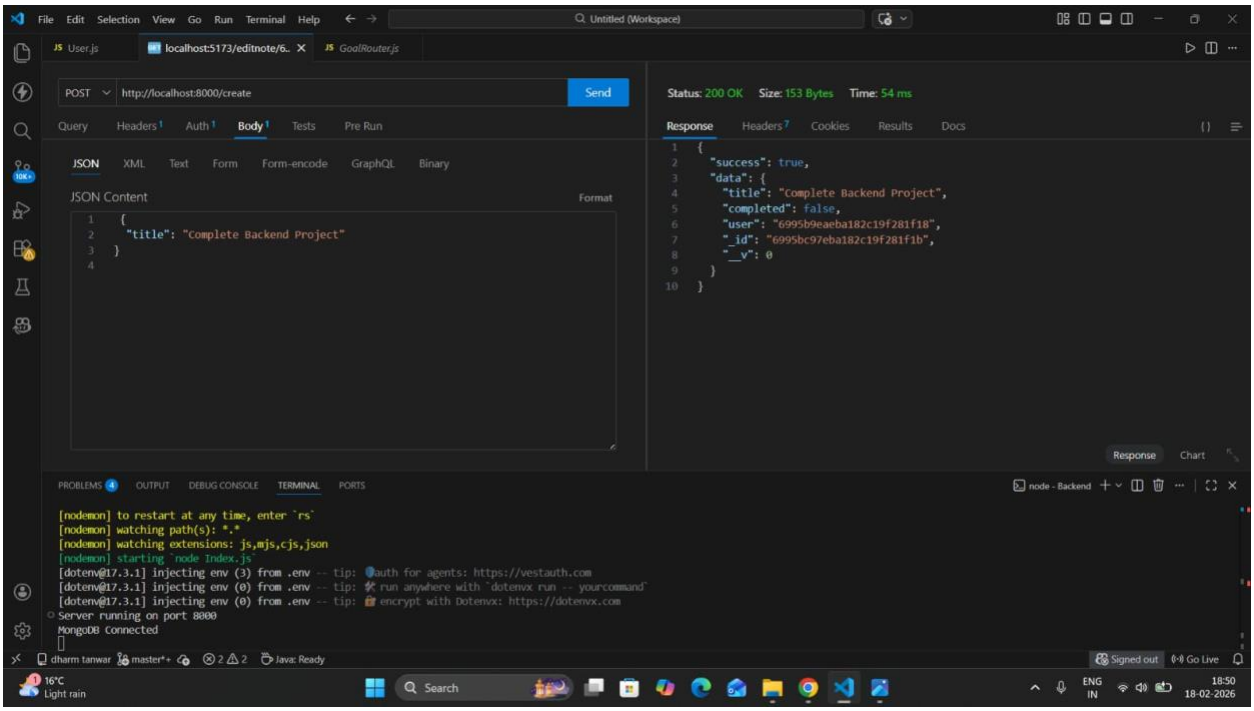
9. Middleware & Security Implementation

Security is enforced at the application level through specialized logic:

JWT Verification: A dedicated middleware intercepts requests to protected routes to validate the JSON Web Token provided in the header.

Payload Validation: Ensures that incoming data for registration and goal creation meets the required schema constraints.

Request Interception: The system validates that the User ID extracted from the token matches the owner of the requested resource.



10. Data Modeling & Relationships

Though using a NoSQL database, the system implements a relational logic:

User Schema: Defines fields for Name, Email (marked as unique), and a Hashed Password.

Goal Schema: Contains the goal title, a boolean for completion status, and a Reference ID pointing back to the User.

One-to-Many Logic: A single user is linked to multiple goal documents, ensuring strict data isolation.

11. Environment Configuration

To maintain system integrity, sensitive information is managed via environment variables:

Database URI: Securely stores the connection string for MongoDB Atlas.

JWT Secret: A private key used by the server to sign and verify digital tokens.

Port Settings: Configuration for the local or production server environment.

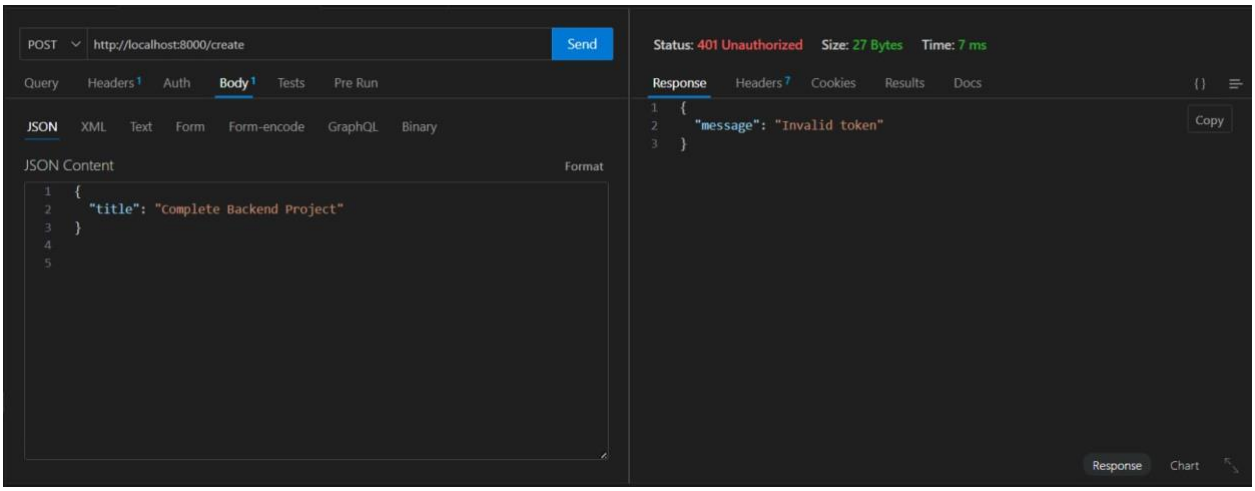
12. Error Handling Framework

The system features a centralized engine to provide consistent API feedback:

Global Catching: A unified error-handling middleware catches all runtime exceptions.

Standardized Responses: Returns a consistent JSON object containing the error message and status code.

Security Focus: Prevents the leakage of sensitive stack traces to the end-user in a production environment.



13. Testing & Quality Assurance

The reliability of the backend was verified through rigorous testing:

Endpoint Testing: All API routes were manually tested using Thunder Client to ensure correct status codes (e.g., 200 OK, 201 Created).

Auth Verification: Testing unauthorized access attempts to ensure the middleware correctly blocks requests without a valid JWT.

CRUD Validation: Confirming that goal updates and deletions only succeed when performed by the verified owner.

14. Scalability & Future Enhancements

The modular MVC architecture allows for future growth of the MicroGoals platform:

Goal Categorization: Adding tags or categories to help users filter objectives more granularly.
Deadline Tracking: Implementing timestamps and due dates for goal urgency.
Social Integration: Allowing users to optionally share goal progress with others while maintaining core privacy.

15. Deployment & Documentation

The project is packaged for professional delivery:
Documentation: This comprehensive guide outlines the architecture and security protocols.
Demonstration: A walkthrough video provides proof of a fully functional registration and management flow.
Repository: Clean, modular folder structures (Models, Routes, Middleware) ensure the code is maintainable by other developers.

```
//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

Conclusion

MicroGoals successfully demonstrates the implementation of a secure, user-centric API through its modular architecture and rigorous authentication protocols. By integrating **JWT-based**

security with a **relational NoSQL approach**, the system ensures that data ownership is strictly enforced and user information remains protected. With a fully functional suite of CRUD operations and a centralized error-handling engine, MicroGoals provides a professional-grade framework for personal productivity. This project stands as a complete, tested, and reliable backend foundation for any modern goal management application.

16. Project Demonstration

Project Demo Link:

https://drive.google.com/file/d/1LsRowBONf_8Rjd9gl2fV9uOc-Uq2HmvR/view?usp=sharing

Demo Link:

https://drive.google.com/file/d/1LsRowBONf_8Rjd9gl2fV9uOc-Uq2HmvR/view?usp=sharing